UNIDAD 3

DISEÑO Y REALIZACIÓN DE PRUEBAS

IES Francisco Ayala

Ciclo Formativo de Grado Medio: ***Desarrollo de Aplicaciones Web***

Módulo: **ENTORNOS DE DESARROLLO**

Contenidos

1. Filosofía de las pruebas del software
2. Estrategia de pruebas del software
3. Técnicas de diseño de casos de prueba
4. Documentación de las pruebas
5. Herramientas de depuración
6. Pruebas automáticas

Introducción

Introducción

- Cuando se desarrolla una **aplicación informática** se desea que sea de **calidad** (responde a las necesidades del cliente) y que tenga el **menor número posible de errores**.

- El **objetivo** de la etapa de pruebas es **descubrir defectos** en la aplicación antes de ser entregada al cliente.

1. Filosofía de las pruebas del software

1. Filosofía de las pruebas del software

- Una de las **tareas del ciclo de vida del software** son las pruebas las cuales permiten detectar defectos cometidos durante el desarrollo del software.
- El objetivo de las pruebas es la detección de defectos y no la demostración de que el software no tiene ninguno.
- En muchos casos resulta **imposible detectar todos los errores** de una aplicación, pero se deben realizar pruebas que **no supongan un esfuerzo excesivo** pero con las que haya una **alta probabilidad de detectar fallos**.

1. Filosofía de las pruebas del software

- Normalmente los propios **desarrolladores** del software son los encargados de **probar sus propios programas**. No obstante, en el caso de **proyectos grandes** y sobre todo en **pruebas de bajo nivel**, se suele involucrar a un **equipo de pruebas independiente** con el que se trabajaría conjuntamente.
- Las pruebas del software consisten en realizar la **validación** y la **verificación** del producto.

1. Filosofía de las pruebas del software

- **VALIDACIÓN**: comprobación de si se está construyendo el producto correcto, es decir, el que desea el cliente y cumple con los requisitos especificados.

- **VERIFICACIÓN**: comprobación de si el producto funciona correctamente, es decir, si lo que hace lo realiza bien y de forma correcta.

1. Filosofía de las pruebas del software

- **Estrategias** que se deben considerar relacionadas con la **puesta en práctica de las pruebas**:

  - **ESTRATEGIA DE APLICACIÓN DE LAS PRUEBAS**:

    Establecer qué partes del software se deben probar.

  - **TÉCNICAS DE DISEÑO DE CASOS DE PRUEBA**:

    Decidir de qué manera se deben realizar, es decir, seleccionar la técnica apropiada.

1. Filosofía de las pruebas del software

- Un **caso de prueba** consiste básicamente en indicar:
  - Qué **datos de entrada** se deben suministrar al programa.
  - Qué **datos de salida** se deben obtener.
  - Cuál es el **fin** de llevar a cabo dicho caso de prueba.

- En caso de que una vez ejecutada la prueba, la salida obtenida no coincida con la salida esperada, esto significa que se ha detectado algún defecto, se realizará la tarea de **depuración**.
- La **depuración** es una tarea resultado de una prueba exitosa en la que se localiza el error y se corrige.

Actividad 3.1. Intervención de diferentes roles en las pruebas del software

Realiza un documento en el que se indique de qué manera intervienen los siguientes roles en las pruebas del software, sus funciones y características deseables:

- Ingeniero de control de calidad/probadores de software.
- Desarrolladores de software.
- Líder/gerentes de proyecto.
- Usuarios finales.
- Ingenieros de sistemas.

El formato del fichero a entregar debe ser ".pdf".

Una vez realizado, se deberá entregar a través de la tarea "*Actividad 3.1. Intervención de diferentes roles en las pruebas del software*" de la Unidad 3, en la plataforma Moodle Centros.

Actividad 3.2. Rentabilidad de las pruebas del software

Realiza un documento en el que se explique cuál es la rentabilidad de las pruebas del software y de qué depende.

El formato del fichero a entregar debe ser “.pdf”.

Una vez realizado, se deberá entregar a través de la tarea “*Actividad 3.2. Rentabilidad de las pruebas del software*” de la Unidad 3, en la plataforma Moodle Centros.

2. Estrategia de pruebas del software

2. Estrategia de pruebas del software

- La pruebas comienzan por **porciones pequeñas de software** y posteriormente se van **incrementando**.
- Se suelen llevar a cabo 4 tipos de pruebas secuenciales:
  - **Prueba de unidad**.
  - **Prueba de integración**.
  - **Prueba del sistema**.
  - **Prueba de validación**.

2. Estrategia de pruebas del software

- **PRUEBA DE UNIDAD**:
  - En el *paradigma estructurado*:
    - Prueba de módulos individuales.
  - En el *paradigma orientado a objetos*:
    - Prueba de clases (prueba de todos sus métodos).

2. Estrategia de pruebas del software

- **PRUEBA DE INTEGRACIÓN**:
  - Comprueba si las clases del software funcionan correctamente cuando cooperan entre ellas.
  - Enfoques para realizar la prueba:
    - **Integración no incremental o big bang**:
      - Se prueban los módulos por separado, se combinan todos y se prueba el programa completo.
      - Se encuentran muchos errores y la corrección es difícil.
    - **Integración incremental**:
      - Se va construyendo el programa completo y probando en pequeños segmentos.
      - Localización de errores más fácil.
      - Estrategias:
        - **Ascendente**: comienza por los módulos de niveles más bajos.
        - **Descendente**: comienza por el programa principal hacia abajo en la jerarquía de control.

- Investigación -
“Ampliación de las Pruebas de Integración”

Elabora un documento con la siguiente información:

- Otros tipos de pruebas de integración incremental (nombre, función y características).
- Gráficos que muestren el funcionamiento de cada tipo de prueba de integración y explicación de dichos gráficos.
- Tabla comparativa con las ventajas y desventajas de todos los tipos de pruebas de integración.

El formato del fichero a entregar debe ser “.pdf”.

Una vez realizado, se deberá entregar a través de la tarea “*Ampliación de las pruebas de Integración*” de la Unidad 3, en la plataforma Moodle Centros.

- Cuestionario -
“Pruebas Unitarias y Pruebas de Integración”

Lee la información disponible en la web:

https://qalified.com/es/blog/pruebas-de-integracion-que-son/

Posteriormente resuelve las preguntas del cuestionario disponible en Moodle.

El formato del fichero a entregar debe ser “.pdf”.

Una vez realizado, se deberá entregar a través de la tarea “*Cuestionario. Pruebas Unitarias y Pruebas de Integración*” de la Unidad 3, en la plataforma Moodle Centros.

2. Estrategia de pruebas del software

- **PRUEBA DEL SISTEMA**:
  - Comprueba la robustez del sistema, es decir, si cumple con los requisitos especificados:
    - Cumplimiento de los requisitos funcionales.
    - Funcionamiento de las interfaces.
    - Adecuación de la documentación de usuario.
    - Ejecución y rendimiento en condiciones límite y de sobrecarga.
    - Tipos de prueba del sistema (no funcionales):
      - **Pruebas de recuperación**: se fuerza un fallo y se comprueba que el sistema se recupera adecuadamente.
      - **Pruebas de seguridad**: se verifica que el sistema está protegido contra accesos ilegales.
      - **Pruebas de esfuerzo o resistencia**: se somete al sistema a situaciones extremas, es decir, en las que se demandan gran cantidad de recursos (memoria, procesamiento, transferencia de datos, etc.) y se comprueba su comportamiento.
      - **Pruebas de rendimiento**: se comprueba que el sistema responda en un tiempo limitado.
      - **Pruebas de despliegue o configuración**: se prueba el sistema en distintas plataformas.

2. Estrategia de pruebas del software

- **PRUEBA DE VALIDACIÓN**:
  - Comprueba si el software es considerado válido por la persona usuaria y si está preparado para su implantación.
  - Los **criterios de validación** deben estar especificados en la **ERS** (especificación de requisitos del software).
  - Tipos de pruebas de validación:
    - **Pruebas Alfa**:
      - Se realizan por el cliente o el usuario en el lugar de desarrollo.
      - El cliente utiliza el software (en muchos casos, todavía no finalizado) de forma natural bajo la observación del desarrollador que irá registrando los errores y problemas de uso.
    - **Pruebas Beta**:
      - Se realizan después de la fase de pruebas alfa.
      - Se realizan por los usuarios finales del software en sus lugares de trabajo.
      - El desarrollador no está presente.
      - El usuario registra todos los problemas que encuentra, e informa al desarrollador, de manera que este realice las modificaciones necesarias y posteriormente prepare una nueva versión del producto.

- Infografía -
“Pruebas funcionales y no funcionales”

Elabora una infografía con CANVA de 2 páginas. En la primera página mostrarás la información de las pruebas funcionales y en la segunda la de las no funcionales.

Para cada tipo de prueba deberás exponer:

- Su nombre
- En qué consiste
- Una imagen representativa
- Una URL de un vídeo/texto/esquema en la que se desarrolle de forma más amplia dicha prueba.

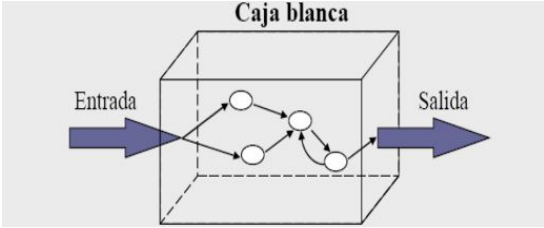
El formato del fichero a entregar debe ser “.pdf”.

Una vez realizado, se deberá entregar a través de la tarea “*Infografía. Pruebas funcionales y no funcionales*” de la Unidad 3, en la plataforma Moodle Centros.

3. Técnicas de diseño de casos de prueba

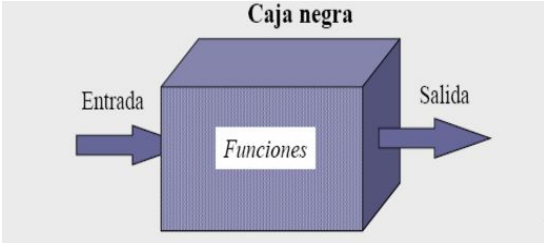
3. Técnicas de diseño de casos de prueba

- **PRUEBAS DE CAJA BLANCA (pruebas de caja de cristal) O PRUEBAS ESTRUCTURALES**:
  - Examinan los detalles de cada módulo, para lo que se debe disponer del **código fuente**.
  - Se prueban los diferentes caminos, los bucles, las variables, etc.



https://jamj2000.gitbooks.io/apuntes-de-entornos-de-desarrollo-1-daw/content/3.PRUEBAS.html

- **PRUEBAS DE CAJA NEGRA O PRUEBAS FUNCIONALES**:
  - Se considera el software como una caja negra que recibe una serie de entradas y proporciona una serie de salidas.
  - No se necesita disponer del código fuente, solo es necesario conocer la **función** que realiza el software.
  - Su objetivo es validar los requisitos funcionales.



https://jamj2000.gitbooks.io/apuntes-de-entornos-de-desarrollo-1-daw/content/3.PRUEBAS.html

Habitualmente se emplean los dos tipos de técnicas, las cuales de pueden combinar y complementar perfectamente.

3. Técnicas de diseño de casos de prueba

- **PRUEBAS DE CAJA BLANCA (pruebas de caja de cristal) O PRUEBAS ESTRUCTURALES**:
  - **PRUEBA DEL CAMINO BÁSICO.**
  - **PRUEBA DE BUCLES.**
  - **PRUEBA DE FLUJO DE DATOS.**
- **PRUEBAS DE CAJA NEGRA O PRUEBAS FUNCIONALES**:
  - **PARTICIONES O CLASES DE EQUIVALENCIA.**
  - **ANÁLISIS DE VALORES LÍMITE.**
  - **CONJETURA DE ERRORES.**

3.1. Pruebas de caja blanca

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

- Esta técnica permite obtener una medida de la complejidad del módulo probado llamada **complejidad ciclomática de McCabe**, que es un número entero que indica el número de caminos independientes que hay en el código y, en consecuencia, el número de casos de prueba que hay que ejecutar para garantizar que se recorren todas las sentencias del código al menos una vez.

- Para calcular esta medida y generar los casos de prueba, es necesario representar el código fuente que se va a probar en un **grafo de flujo**.

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

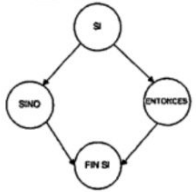
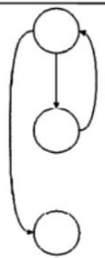
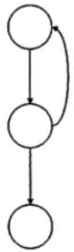
¿Cómo se crean grafos de flujo?

- Se asigna un número único a cada sentencia del código y a cada condición. Si se dispone de varias instrucciones en secuencia, se puede asignar un número único a todas ellas y estas serán tratadas como un bloque. Cada uno de estos elementos constituirá un **nodo**, representado por un círculo en cuyo interior aparecerá el número asignado.
- Se unen los nodos mediante flechas llamadas **aristas** que representan el flujo de control.
- Las áreas delimitadas por aristas y nodos se llaman **regiones**. El área exterior del grafo es otra región más.

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

Representación de las estructuras básicas de programación en un grafo de flujo:

ESTRUCTURA	GRAFO DE FLUJO
SECUENCIAL Instrucción 1 Instrucción 2 ..... Instrucción n	
CONDICIONAL **Si** <condición> **Entonces** <Instrucciones> **Si no** <Instrucciones> **Fin si**	
HACER MIENTRAS **Mientras** <condición> **Hacer** <instrucciones> **Fin mientras**	
REPETIR HASTA **Repetir** <instrucciones> **Hasta que** <condición>	

Entornos de desarrollo. Alicia Ramos, María Jesús Ramos. Garceta. 2014

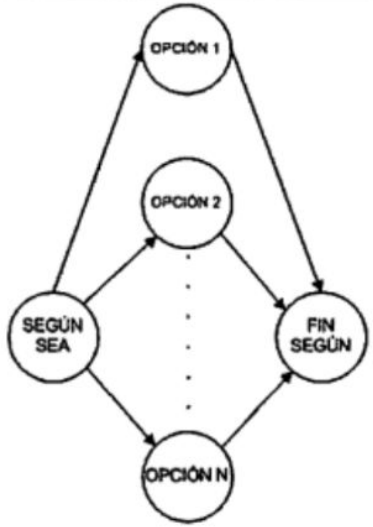
3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

Representación de las estructuras básicas de programación en un grafo de flujo:

CONDICIONAL MÚLTIPLE

```
Según  sea <variable> Hacer
     Caso  opción 1:
                <Instrucciones>
     Caso  opción 2:
                <Instrucciones>
     Caso  opción  3:
                <Instrucciones>
     Otro caso:
                <Instrucciones>
 Fin según
```

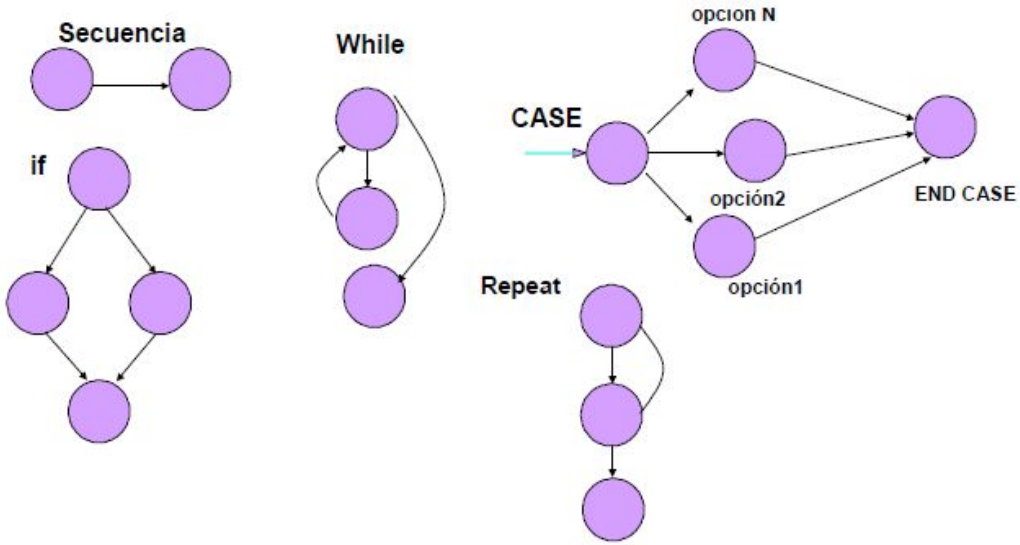


Entornos de desarrollo. Alicia Ramos, María Jesús Ramos. Garceta. 2014

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

Representación de las estructuras básicas de programación en un grafo de flujo:



https://www.cartagena99.com/recursos/alumnos/apuntes/210927125051-Videotutoria%204%20Grafos.pdf

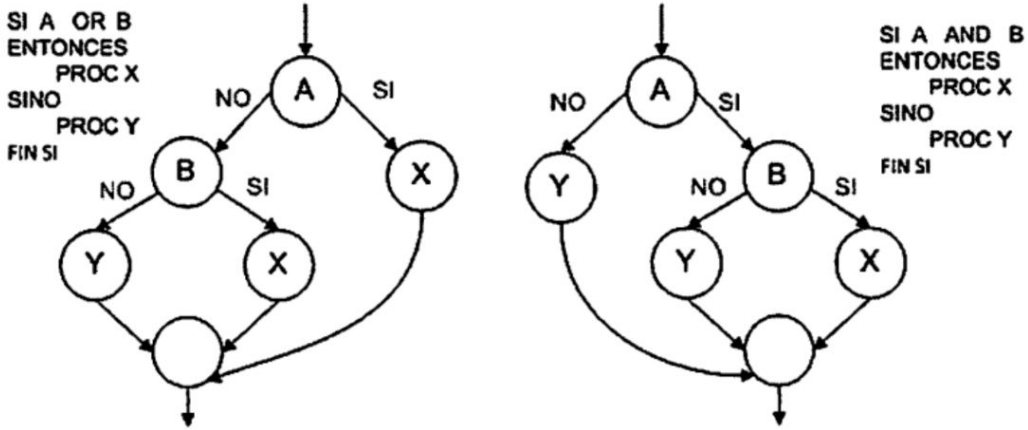
- Nodos: Representan cero, una o varias sentencias.
- Aristas: líneas que unen dos nodos.
- Regiones: áreas delimitadas por aristas y nodos. Cuando se contabilizan las regiones de un programa debe incluirse el área externa como una región más

https://docplayer.es/18156212-Clase-5-tecnicas-de-caja-blanca.html

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

Representación de las estructuras básicas de programación (condiciones compuestas) en un grafo de flujo:



Nodos Predicado: Cuando en una condición aparecen uno o más operadores lógicos (AND, OR, XOR, ...) se crea un nodo distinto por cada una de las condiciones simples. Cada nodo generado de esta forma se denomina nodo predicado.

https://docplayer.es/18156212-Clase-5-tecnicas-de-caja-blanca.html

Entornos de desarrollo. Alicia Ramos, María Jesús Ramos. Garceta. 2014

EJEMPLOS DE REALIZACIÓN DE GRAFOS DE FLUJO

SÍMBOLOS EN UN DIAGRAMA DE FLUJO

INICIO / FINAL: Se utilizan para identificar el inicio y el final del diagrama.

ENTRADA DE DATOS: Indica una entrada de datos en el proceso pro cualquier dispositivo

ENTRADA DE DATOS POR TECLADO: Datos que se intoducen por teclado. El ordenador está esperando que un operario introduzca un dato

ACCIÓN/PROCESO: Indica una acción que debe realizarse.

FLUJO / FLECHAS DE DIRECCIÓN: Indica el seguimiento lógico del diagrama

DECISIÓN: Indica el seguimiento lógico del diagrama

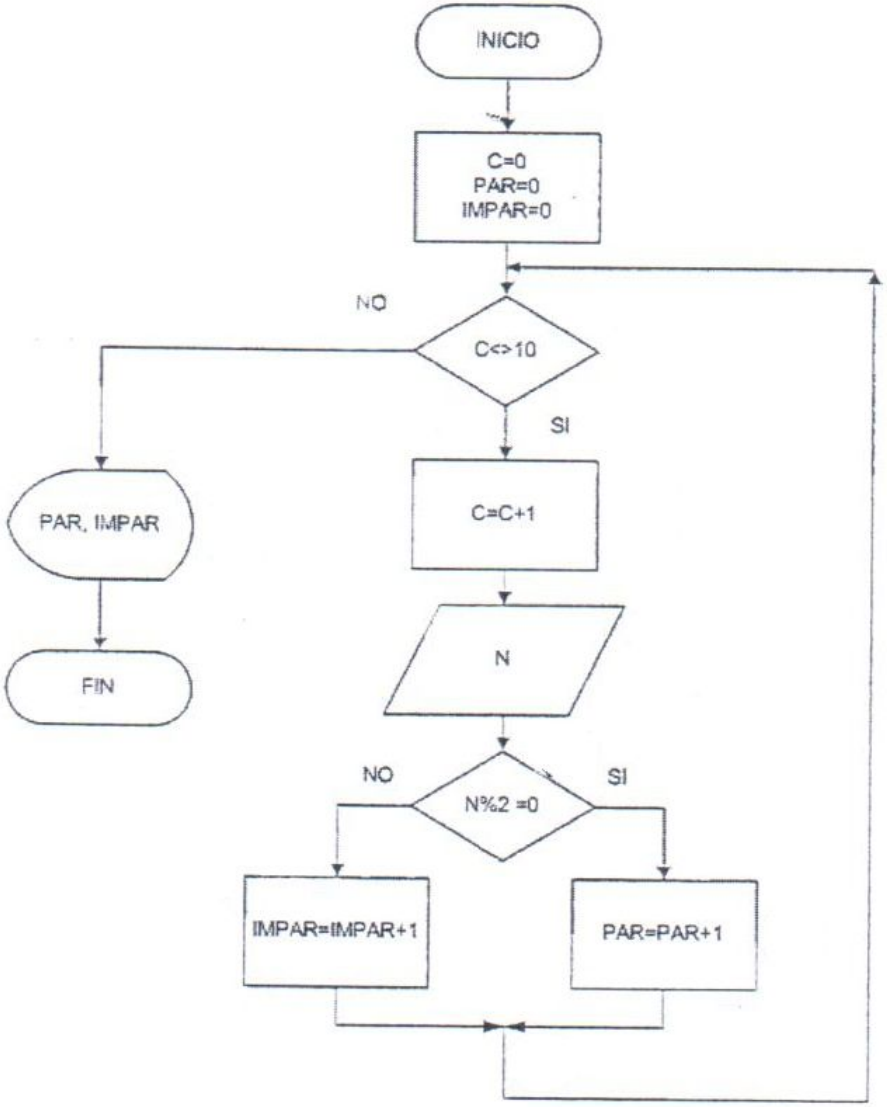
IMPRESORA/DOCUMENTO: Indica la presentación de uno o varios resultados de forma impresa

PANTALLA: Instrucción de presentación de mensajes o resultados en pantalla.

https://sgmspain.wordpress.com/tag/diagrama-de-flujo/

EJEMPLO 1:

Programa que lee 10 números de teclado y muestra cuántos de los números leídos son pares y cuántos son impares.



EJEMPLO 2:

Se dispone de un fichero de Alumnos con la siguiente estructura de registro: Curso, Nombre, Sexo (puede ser H o M) y Nota. El fichero está ordenado ascendentemente por curso. Vamos a realizar un proceso que lea los registros del fichero y muestre
por cada curso el número de hombres y el número de mujeres.

```
Inicio
  Abrir Fichero Alumnos
  Leer Alumnos (Curso, Nombre, Sexo, Nota)
  Mientras  Haya registros  Hacer
    NH = 0, NM = 0
    Visualizar Curso
    Mientras  Haya registros  y  Mismo curso  Hacer
      Si  sexo = "H"   Entonces
        NH = NH + 1
      Sino
        NM = NM + 1
      Fin si
      Leer Alumnos (Curso, Nombre, Sexo, Nota)
    Fin Mientras
    Visualizar NH, NM
  Fin Mientras
  Cerrar Fichero Alumnos
Fin
```

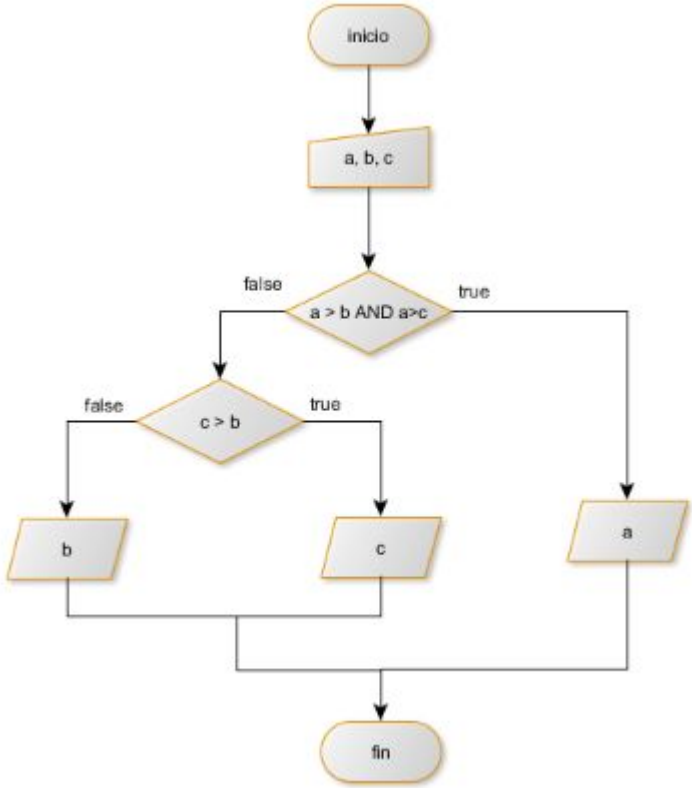
EJERCICIOS

En la plataforma Moodle Centros se encuentran disponibles los diferentes enunciados.

No habrá que hacer la entrega de dichos ejercicios.

EJERCICIO 1:

Realiza el grafo de flujo para el siguiente programa que determina el número mayor de 3 valores dados.



EJERCICIO 2:

Realiza el grafo de flujo para el siguiente programa que determina el número de caracteres que tiene una cadena dada.

```
int contar_letra(char cadena[10], char letra)
{
  int contador, n,  lon;
  n=0; contador=0;
  lon = strlen (cadena);
  if (lon > 0) {
              do {
                if (cadena[contador] == letra) n++;
                contador++;
                lon--;
              } while (lon > 0);
  }
  return n;
}
```

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

Una vez creado el grafo de flujo, se puede calcular la **complejidad ciclomática de McCabe V(G)** de cualquiera de las tres formas siguientes:

- V(G) = número de regiones
- V(G) = A - N + 2
- V(G) = NP + 1

A: *número de aristas*

N: *número de nodos*

NP: *número de nodos predicado*

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

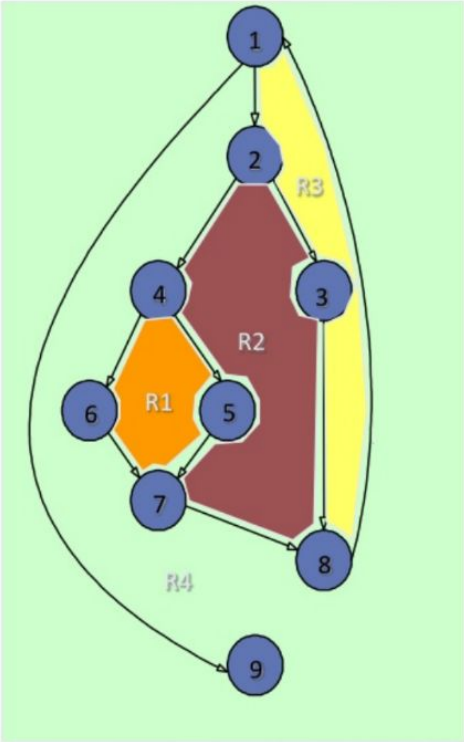
Se establecen los siguientes valores de referencia de la complejidad ciclomática:

Complejidad ciclomática	Evaluación de riesgo
Entre 1 y 10	Programas o métodos sencillos, sin mucho riesgo.
Entre 11 y 20	Programas o métodos más complejos, riesgo moderado.
Entre 21 y 50	Programas o métodos complejos, alto riesgo.
Mayor que 50	Programas o métodos no testeables, muy alto riesgo.

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

- V(G) = 4
  - El grafo de la figura delimita cuatro regiones.
  - 11 aristas - 9 nodos + 2 = 4
  - 3 nodos predicado + 1 = 4



3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

El valor **V(G)** indica el **número de caminos independientes** del grafo de flujo y, por tanto, el **número de casos de prueba** que hay que generar.

Un **camino independiente** es cualquier camino del programa que introduce, **por lo menos, un *nuevo conjunto de sentencias de proceso o una condición***. En un diagrama de flujo, un camino independiente está constituido ***por lo menos por una arista que no haya sido recorrida*** anteriormente a la definición del camino.

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

Determinar el conjunto básico de caminos independientes

Un camino independiente es cualquier camino del programa que introduce, por lo menos, un nuevo conjunto de sentencias de proceso o una condición, respecto a los caminos existentes. En términos del diagrama de flujo, un camino independiente está constituido por lo menos por una arista que no haya sido recorrida anteriormente a la definición del camino. En la identificación de los distintos caminos de un programa para probar se debe tener en cuenta que cada nuevo camino debe tener el mínimo número de sentencias nuevas o condiciones nuevas respecto a los que ya existen. De esta manera se intenta que el proceso de depuración sea más sencillo.

El conjunto de caminos independientes de un grafo no es único. No obstante, a continuación, se muestran algunas heurísticas para identificar dichos caminos:

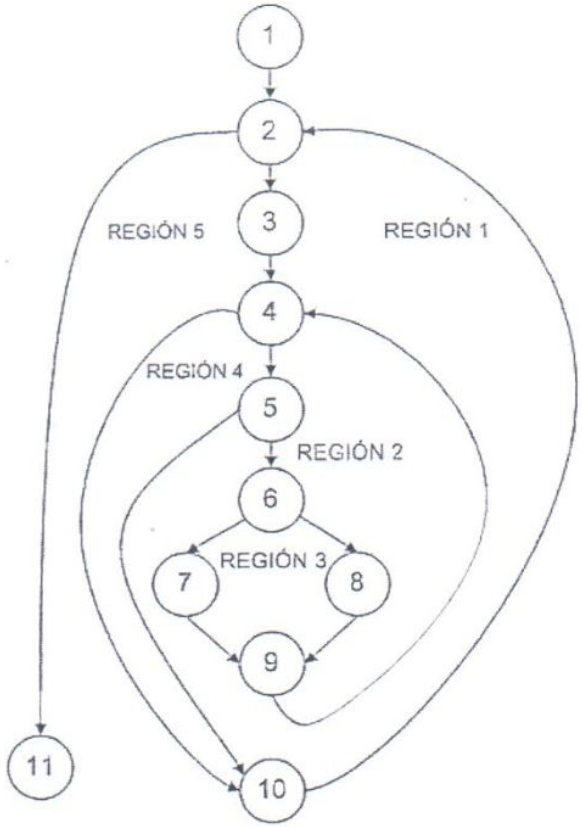
(a) Elegir un *camino principal* que represente una función válida que no sea un tratamiento de error. Debe intentar elegirse el camino que atraviese el máximo número de decisiones en el grafo.

(b) Identificar el segundo camino mediante la localización de la *primera decisión* en el camino de la línea básica alternando su resultado mientras se mantiene el máximo número de decisiones originales del camino inicial.

(c) Identificar un tercer camino, colocando la primera decisión en su valor original a la vez que se altera la *segunda decisión* del camino básico, mientras se intenta mantener el resto de decisiones originales.

(d) Continuar el proceso hasta haber conseguido *tratar todas las decisiones*, intentando mantener como en su origen el resto de ellas.

Ejemplo
Caminos independientes



Para el *Ejemplo 2*, la complejidad ciclomática es 5.

1. V(G) = Número de regiones del grafo = 5.
2. V(G) = Aristas – Nodos + 2 = 14 – 11 + 2 = 5
3. V(G) = Nodos predicado + 1 = 4 + 1 = 5

Para el *Ejemplo 2*, un conjunto de caminos independientes será:

- Camino 1: 1 – 2 – 11
- Camino 2: 1 – 2 – 3 – 4 – 10 – 2 – 11
- Camino 3: 1 – 2 – 3 – 4 – 5 – 10 – 2 – 11
- Camino 4: 1 – 2 – 3 – 4 – 5 – 6 – 7 – 8 – 4 – 10 – 2 – 11
- Camino 5: 1 – 2 – 3 – 4 – 5 – 6 – 7 – 9 – 4 – 10 – 2 – 11

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DEL CAMINO BÁSICO

El último paso de este método es **generar casos de prueba** que recorran cada uno de los **caminos** eligiendo los datos de entrada que fuercen cada uno de los mismos.

Puede ocurrir que alguno de los caminos no se pueda recorrer de forma independiente, sino como parte de otro camino.

Para cada caso de prueba, se deben **comparar los resultados obtenidos con los esperados**. Existirá algún **defecto** en caso de que no haya coincidencia, en cuyo caso se llevará a cabo un proceso de **depuración**.

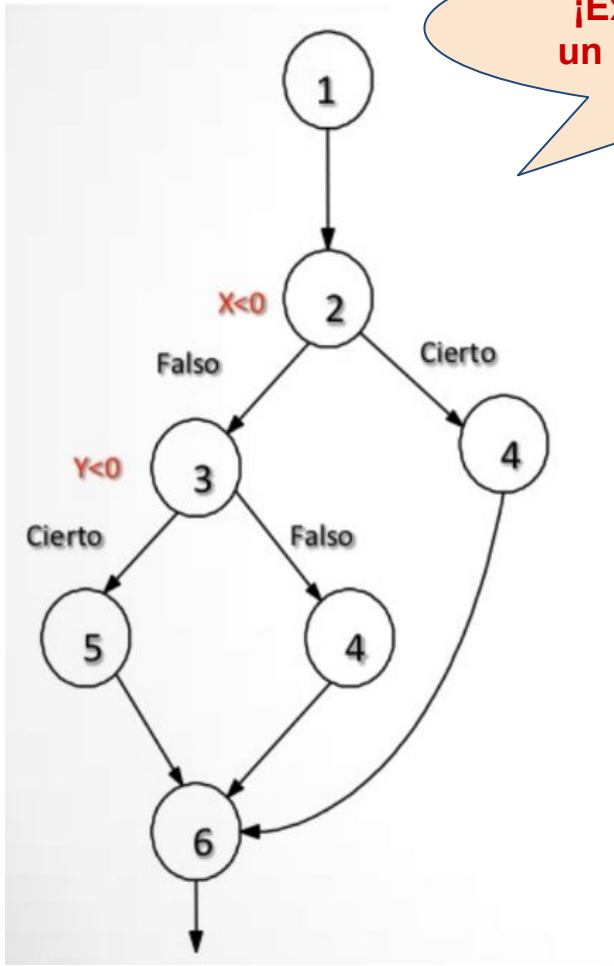
EJERCICIO 1 COMPLETO (Prueba del camino básico)

```
static void visualizarMedia(float x, float y) {
  float resultado = 0;
  if ( x < 0 || y < 0 )
    System.out.println("X e Y deben ser positivos");
  else {
    resultado = (x + y) / 2;
    System.out.println("La media es: " + resultado);
  }

}
```

```
static void visualizarMedia(float x, float y) {
  float resultado = 0;   (1)
  if ( x < 0 || y < 0 )   (3)
(2)   System.out.println("X e Y deben ser positivos");   (4)
  else {
    resultado = (x + y) / 2;
    System.out.println("La media es: " + resultado);   (5)
  }
  (6)
}
```

¡Existe otra solución con un grafo más simplificado!



V(G) = 3 regiones. Por lo tanto, hay que determinar tres caminos independientes.

- Camino 1: 1-2-3-5-6
- Camino 2: 1-2-4-6
- Camino 3: 1-2-3-4-6

Casos de prueba para cada camino:

Camino 1: x=3, y= 5, rdo=4

Camino 2: x=-1, y=3, rdo=0, error

Camino 3: x=4, y=-3, rdo=0, error

```
static void visualizarMedia(float x, float y) {
  float resultado = 0;   (1)
  if ( x < 0 || y < 0 )   (3)
(2)  System.out.println("X e Y deben ser positivos");   (4)
  else {
    resultado = (x + y) / 2;
    System.out.println("La media es: " + resultado);   (5)
  }
  (6)
}
```

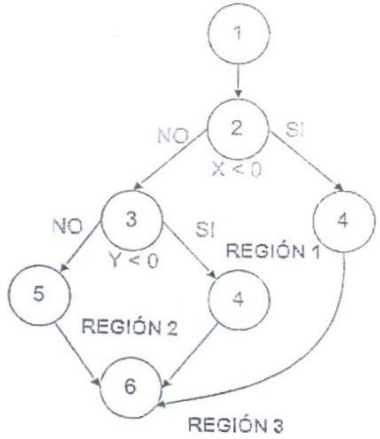


Figura 3.9. Ejemplo 3.

Los caminos independientes y los casos de prueba para cada camino se muestran en la siguiente tabla:

Camino	Caso de prueba	Resultado esperado
Camino 1: 1-2-3-5-6	Escoger algún X e Y tal que NO se cumpla la condición X<0 || Y<0 X=4, Y=5 visualizarMedia(4,5)	Visualiza: La media es: 4.5
Camino 2: 1-2-4-6	Escoger algún X tal que SÍ se cumpla la condición X<0 (Y puede ser cualquier valor) X=-4, Y=5 visualizarMedia(-4,5)	Visualiza: X e Y deben ser positivos
Camino 3: 1-2-3-4-6	Escoger algún X tal que NO cumpla la condición X<0 y escoger algún Y que SÍ cumpla la condición Y<0 X=4, Y=-5 visualizarMedia(4,-5)	Visualiza: X e Y deben ser positivos

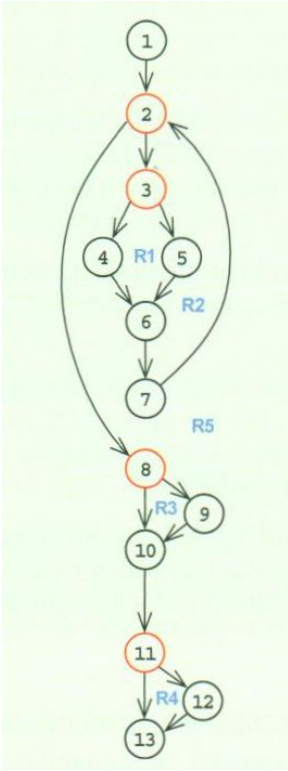
EJERCICIO 2 COMPLETO (Prueba del camino básico)

```
begin
        int num=0, cont_pos=0, cont_neg=0, suma_pos=0, suma_neg=0;
        float media_pos=0, media_neg=0;
        System.out.print ("Introduce número: ");
        num=Entrada.entero();

        while (num != 0)
                if (num > 0)
                        cont_pos++;
                        suma_pos+=num;
                else
                        cont_neg++;
                        suma_neg+=num;
                endif;
                System.out.print ("Introduce número: ");
                num=Entrada.entero();
        endwhile;
        if (cont_pos != 0)
                media_pos = (float)suma_pos/cont_pos;
                System.out.println("Media de los positivos: " + media_pos);
        endif;
        if (cont_neg != 0)
                media_neg = (float)suma_neg/cont_neg;
                System.out.println ("Media de los negativos: " + media_neg);
        endif;
end;
```

```
begin
    int num = 0, cont_pos=0, cont_neg=0, suma_pos=0, suma_neg=0;
1   float media_pos=0,media_neg=0;
    System.out.print("Introduce número: ");
    num=Entrada.entero();
2   while (num != 0)
3       if (num > 0)
4           cont_pos++;
            suma_pos+=num;
        else
5           cont_neg++;
            suma_neg+=num;
        endif;
6       System.out.print("Introduce número: ");
        num=Entrada.entero();
7   endwhile;
8   if (cont_pos ! = 0)
9       media_oos = (float)suma_pos/cont_pos;
        System.out.println("Media de los positivos: "+ media_pos);
10  endif;
11  if (cont_neg !=0)
12      media_neg= (float)suma_neg/cont_neg;
        System.out.println("Media de los negativos: "+ media_neg);
13  endif;
end;
```

¡Existe otra solución con un grafo más simplificado!



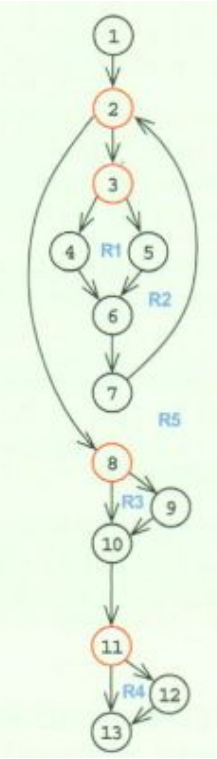
Una vez que se tiene el grafo de flujo, ya se puede calcular la complejidad ciclomática, *V(G)*, de las tres formas que se ha indicado antes:

V(G) = número de regiones = 5

$V(G) = A - N + 2 = 16 - 13 + 2 = 5$

$V(G) = NP + 1 = 4 + 1 = 5$

Como *V(G)* toma el valor 5, se deduce que hay cinco caminos independientes en el código examinado y, por tanto, el número de casos de prueba que hay que crear para cubrir todas las instrucciones y condiciones del programa es 5. La siguiente tarea es encontrar esos cinco caminos independientes. Para ello, hay que tener en cuenta que cada camino independiente incluye una ruta nueva en relación con los anteriores:



Entornos de desarrollo. José Manuel Piñeiro Gómez. Paraninfo. 2022

Camino 1: 1-2-8-10-11-13

Camino 2: 1-2-3-4-6-7-2-8-10-11-13

Camino 3: 1-2-3-4-6-7-2-8-9-10-11-13

Camino 4: 1-2-3-5-6-7-2-8-10-11-13

Camino 5: 1-2-3-5-6-7-8-10-11-12-13

Con objeto de probar el camino 1, se debe introducir como primer número el 0 con el fin de que no se entre en el bucle ni una sola vez y se pase directamente del nodo 2 al nodo 8. En este caso, el programa no debe mostrar nada, ya que no se ha introducido ningún número ni positivo, ni negativo.

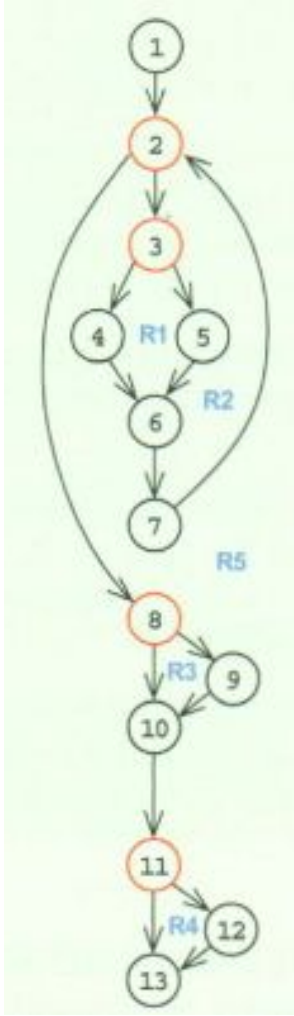
El camino 2 no se puede probar como tal, ya que al introducir un número positivo (para pasar por el nodo 4), se debería pasar por el nodo 9, que muestra la media de los números positivos. Por consiguiente, este camino debe probarse como parte de otro.

Para probar el camino 3, hay que suministrar al programa, en primer lugar, un número positivo (para pasar por el nodo 4), por ejemplo, el número 8, y un 0 a continuación (para no pasar más por el bucle). El programa deberá indicar que la media de los números positivos es 8. Al probar el camino 3, ya se prueba el camino 2 como parte de él.

El camino 4 tampoco se puede probar como tal, pues al introducir un número negativo (para pasar por el nodo 5), se debe pasar por el nodo 12 para mostrar la media de los números negativos.

Por último, a fin de probar el camino 5, se debe suministrar al programa un número negativo (para pasar por el nodo 5), por ejemplo, el –4, y un 0 a continuación. El programa deberá indicar que la media de los números negativos es –4. Al probar el camino 5, se prueba también el camino 4 como parte de él.

Para cada caso de prueba, se comprueba que el resultado mostrado coincide con el esperado, de forma que la no coincidencia indicará la existencia de algún defecto, por lo que habrá que proceder a detectar y corregir dicho error (mediante un proceso de depuración).



3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DE BUCLES

La prueba de bucles se centra exclusivamente en las **estructuras repetitivas o bucles**.

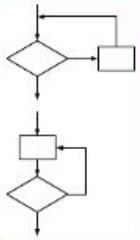
Los casos de prueba que se deben generar para cada tipo de bucle son:

- **Bucles simples**: si el número máximo de iteraciones por el bucle es *n*, se deben probar los siguientes casos:
  - Saltarse completamente el bucle.
  - Realizar una iteración por el bucle.
  - Realizar dos iteraciones por el bucle.
  - Realizar *m* iteraciones por el bucle, con m < n.
  - Realizar *n*-1, *n* y *n*+1 iteraciones por el bucle.
- **Bucles anidados**: se deben llevar a cabo los siguientes pasos:
  - Comenzar por el bucle más interno, estableciendo valores mínimos para todos los demás bucles.
  - Realizar pruebas de bucle simple para el bucle más interno, manteniendo los bucles exteriores en sus valores mínimos.
  - Trabajar de fuera hacia dentro y probar el siguiente bucle manteniendo los otros bucles exteriores en valores mínimos y los otros bucles interiores en valores típicos.
  - Continuar hasta que se hayan probado todos los bucles.
- **Bucles concatenados**: si los bucles son independientes unos de otros, se pueden probar empleando la estrategia indicada para los bucles simples. Si los bucles concatenados son dependientes porque el segundo bucle usa el contador del primer bucle como valor inicial para el segundo, se recomienda emplear el enfoque aplicado a los bucles anidados.

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DE BUCLES

Bucles Simples

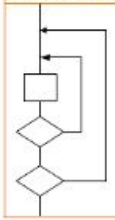


Se debe aplicar:

- Pasar por alto totalmente el bucle
- Pasar una sola vez por el bucle
- Pasar dos veces por el bucle
- Hacer m pasos por el bucle con m < n
- Hacer n-1, n y n+1 pasos por el bucle

Donde n, es el número máximo de pasos permitidos por el bucle.

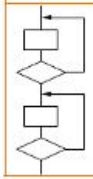
Bucles Anidados



Se debe:

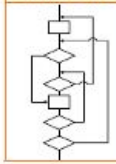
- Comenzar por el bucle más interior
- Llevar a cabo las pruebas de bucles simples para el bucle más interior
- Progresar hacia fuera, llevando a cabo pruebas para el siguiente bucle
- Continuar hasta cuando se prueben todos los bucles.

Bucles Concatenados



Se pueden probar con el método para buques simples, siempre y cuando los bucles sean independientes.
Cuando los bucles no son independientes se utiliza el enfoque para bucles anidados.

Bucles no estructurados



Estos bucles se deben rediseñar.

https://ingsoftware2020.webcindario.com/tercera-unidad/tecnicas-de-prueba-del-software/prueba-de-la-estructura-de-control.html

PRUEBA DE BUCLES

EJEMPLOS DE BUCLES SIMPLES

```
int i;

for (i = 0; i <= 10; i++) //Sentencia For
{
    System.out.println(i); //instrucciones del bucle
}
```

```
int i=1;
        while (i < 100){
        i++;
        System.out.println(""+i);
    }
```

PRUEBA DE BUCLES

EJEMPLOS DE BUCLES ANIDADOS

```
//1
//12
//123
//1234
//12345
public class ProgramaConBreak {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 5; fila++) {
            for (int columna = 1; columna <= 5; columna++) {
                if (fila < columna) {
                    break;
                }
                System.out.print(columna);
            }
            System.out.println();
        }
    }
}
```

PRUEBA DE BUCLES

EJEMPLO DE BUCLES CONCATENADOS

INDEPENDIENTES

```
// Bucle 1
    for (int i = 1; i <= 3; i++) {
        System.out.println("Bucle 1: " + i);
    }

    // Bucle 2
    for (int j = 1; j <= 2; j++) {
        System.out.println("Bucle 2: " + j);
    }
```

DEPENDIENTES

```
// Bucle 1
for (int i = 1; i <= 3; i++) {
    System.out.println("Bucle 1: " + i);
}

// Bucle 2 dependiente del resultado del Bucle 1
for (int j = i; j <= 11; j++) {
    System.out.println("Bucle 2: " + j);
}
```

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DE FLUJO DE DATOS

La prueba de flujo de datos selecciona caminos de un programa de acuerdo con las ubicaciones de las definiciones y usos de variables en dicho programa.

Aquí hay que considerar que en una **instrucción**:

- Se define una variable si se le asigna valor.
- Se usa una variable si se consulta o utiliza su valor de alguna forma (en una condición, siempre se usan una o varias variables).

Si se asigna el número I a una instrucción, el conjunto ***DEF*** para esa instrucción I estará formado por el conjunto de variables que se definen en esa instrucción. Su conjunto ***USO*** estará formado por el conjunto de variables cuyo valor se usa en esa instrucción. Esto se ilustraría de la siguiente manera:

$$\textbf{\textit{DEF (I)}} = \{ X \mid I \text{ contiene una definición de } X \}$$

$$\textbf{\textit{USO (I)}} = \{ X \mid I \text{ contiene un uso de } X \}$$

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DE FLUJO DE DATOS

La definición de una variable *X* en la instrucción *I* está viva en la instrucción *I'* si hay un camino entre las instrucciones *I* e *I'* que no contiene ninguna otra definición de *X*.

Mediante esta técnica, se encuentran las **cadenas DU** (**cadenas de definición-uso**) para una variable, en las que primero se define una variable y luego se usa esa misma variable sin que haya entre la instrucción donde se define y la instrucción donde se usa ninguna otra definición de la variable.

Técnicamente, estas cadenas tienen la forma *[X, I, I']*, donde *X* es una variable, *I* e *I'* son instrucciones tal que *X* está en *DEF (I)* y en *USO (I')*, y la definición de *X* en *I* está viva en *I'* porque existe algún camino entre *I* e *I'* que no incluye ninguna otra definición de *X*.

Como el objetivo es encontrar cadenas DU, esta técnica también recibe el nombre de **prueba DU**.

3. Técnicas de diseño de casos de prueba. Pruebas de caja blanca

PRUEBA DE FLUJO DE DATOS

Esta técnica requiere llevar a cabo los siguientes pasos:

1. Asignar un número a cada instrucción y condición de un programa.
2. Calcular el conjunto DEF y USO para cada variable que se usa en ese programa.
3. Encontrar las cadenas DU para todas las variables.
4. Generar el número mínimo de casos de prueba que incluyan caminos para todas las cadenas DU.
5. Generar datos de entrada que fuercen cada uno de los caminos encontrados en el paso anterior.

PRUEBA DE FLUJO DE DATOS

El primer paso ya está hecho porque ya se ha asignado un número a cada instrucción y condición.

Seguidamente, se calculan las cadenas DU solo para la variable *num*. A tal fin, se definen los conjuntos *DEF* y *USO* para esta variable. Como se puede observar, la variable *num* se define en las instrucciones 1 y 6, mientras que se usa en las que tienen asignados los números 2, 3, 4 y 5. Esto se indica del siguiente modo:

DEF(1) = {num}

USO(2) = {num}

USO(3) = {num}

USO(4) = {num}

USO(5) = {num}

DEF(6) = {num}

El siguiente paso es buscar las cadenas DU para esta variable, que son las siguientes numeradas:

DU [num, 1, 2] (1)

DU [num, 1, 3] (2)

DU [num, 1, 4] (3)

DU [num, 1, 5] (4)

DU [num, 6, 2] (5)

DU [num, 6, 3] (6)

DU [num, 6, 4] (7)

DU [num, 6, 5] (8)

A continuación, se debe encontrar el número mínimo de caminos que incluyan todas las cadenas DU. Estos caminos son:

```
   begin
      int num = 0, cont_pos=0, cont_neg=0, suma_pos=0, suma_neg=0;
1     float media_pos=0,media_neg=0;
      System.out.print("Introduce número: ");
      num=Entrada.entero();
2     while (num != 0)
3        if (num > 0)
4           cont_pos++;
            suma_pos+=num;
         else
5           cont_neg++;
            suma_neg+=num;
         endif;
6        System.out.print("Introduce número: ");
         num=Entrada.entero();
7     endwhile;
8     if (cont_pos ! = 0)
9        media_oos = (float)suma_pos/cont_pos;
         System.out.println("Media de los positivos: "+ media_pos);
10    endif;
11    if (cont_neg !=0)
12       media_neg= (float)suma_neg/cont_neg;
         System.out.println("Media de los negativos: "+ media_neg);
13    endif;
   end;
```

PRUEBA DE FLUJO DE DATOS

DU [num, 1, 2] (1)

DU [num, 1, 3] (2)

DU [num, 1, 4] (3)

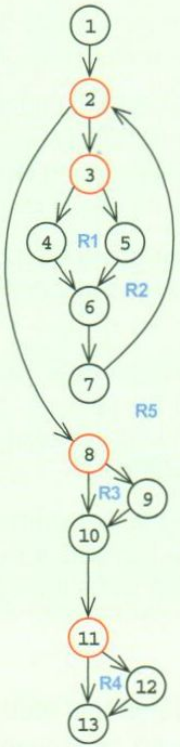
DU [num, 1, 5] (4)

DU [num, 6, 2] (5)

DU [num, 6, 3] (6)

DU [num, 6, 4] (7)

DU [num, 6, 5] (8)



- Camino 1: 1-2-3-4-6-7-2-3-4-6-7-2-3-5-6-7-2-8-9-10-11-12-13. Este camino cubre las cadenas DU: (1), (2), (3), (5), (6), (7) y (8).
- Camino 2: 1-2-3-5-6-7-2-8-10-11-12-13. Este camino cubre la cadena DU (4).

En este caso, con dos caminos es suficiente para cubrir todas las cadenas DU encontradas. En la primera iteración del bucle del camino 1, se cubren las cadenas DU (1), (2) y (3); en la segunda iteración, se cubren las cadenas DU (5), (6) y (7) y, en la tercera iteración, se cubre la cadena DU (8). Pero, en el camino 1, no es posible cubrir la cadena DU con el número (4), es decir, DU [*num*, 1, 5] porque por el nodo 1 solo se pasa una vez. En las iteraciones, no se puede pasar por los nodos 4 y 5 a la vez y, para volver a uno de los nodos 4 o 5, hay que pasar por el nodo 6, que contiene otra definición de la variable *num*. Por este motivo, hay que establecer otro camino para cubrir esta cadena DU.

Ya solo quedaría generar los datos de entrada para recorrer cada uno de estos dos caminos. Para el camino 1, habría que introducir en el programa dos números positivos, luego uno negativo y un cero para finalizar. Por ejemplo, si se suministra un 4, un 8 y un –9, el programa nos debe indicar que la media de los números positivos es 6 y la de los negativos, –9. Para forzar el camino 2, habría que introducir en el programa un número negativo y un cero para finalizar. Si, por ejemplo, se proporciona como entrada el número –10 y luego un 0, el programa debe indicar que la media de los números negativos es –10.

```
   begin
      int num = 0, cont_pos=0, cont_neg=0, suma_pos=0, suma_neg=0;
1     float media_pos=0,media_neg=0;
      System.out.print("Introduce número: ");
      num=Entrada.entero();
2     while (num != 0)
3        if (num > 0)
4           cont_pos++;
            suma_pos+=num;
         else
5           cont_neg++;
            suma_neg+=num;
         endif;
6        System.out.print("Introduce número: ");
         num=Entrada.entero();
7     endwhile;
8     if (cont_pos ! = 0)
9        media_oos = (float)suma_pos/cont_pos;
         System.out.println("Media de los positivos: "+ media_pos);
10    endif;
11    if (cont_neg !=0)
12       media_neg= (float)suma_neg/cont_neg;
         System.out.println("Media de los negativos: "+ media_neg);
13    endif;
   end;
```

Entornos de desarrollo. José Manuel Piñeiro Gómez. Paraninfo. 2022

EJERCICIO FINAL

(Pruebas de caja blanca)

El siguiente programa escrito en Java pide un número entero por teclado e indica si es un número perfecto, es decir, si es igual a la suma de sus divisores propios. Un divisor propio es un entero positivo distinto del número en sí mismo y que divide al número de forma exacta, es decir, sin resto.

```
public static void main(String[] args) {
   int num, divisor, sumadivisores;
   divisor = 1;
   sumadivisores = 0;                                          1
   Scanner entrada = new Scanner(System.in);
   System.out.print ("Introduzca un número mayor que 0: ");
   num = entrada.nextInt();
   while (divisor <= num/2)
   {   if (num % divisor == 0)
          sumadivisores = sumadivisores + divisor;
       divisor++;
   }
   if (num == sumadivisores)
      System.out.println ("El número " + num + " es un número
      perfecto");
   else
      System.out.println ("El número " + num + " no es un número
      perfecto");
}
```

Construye el grafo de flujo para este programa y calcula su complejidad ciclomática. Además, indica un conjunto de caminos independientes y, para cada camino, el caso de prueba correspondiente, incluyendo la entrada proporcionada y la salida esperada.

obtén las cadenas DU para la variable *sumadivisores* y encuentra el número mínimo de caminos de prueba que recorren todas estas cadenas.

3.2. Pruebas de caja negra

CONTINUARÁ EN PARTE 2 …